SQL PROJECT DOCUMENTATION

Book Sales Analysis

Using PostgreSQL

Complete original SQL script with short explanations: table creation, data import, 11 basic and 11 advanced questions.

3	500	22	25
CSV files	rows per table	SQL questions	answer queries

TECHNOLOGY	SOURCE DATA	STRUCTURE	WORKFLOW
PostgreSQL, SQL	**Books.csv, Customers.csv, Orders.csv**	**Part 1: Basic Analysis | Part 2: Advanced Analysis**	**CSV files > PostgreSQL tables > Basic > Advanced queries**

All SQL in this document is reproduced from the original project script without modification.

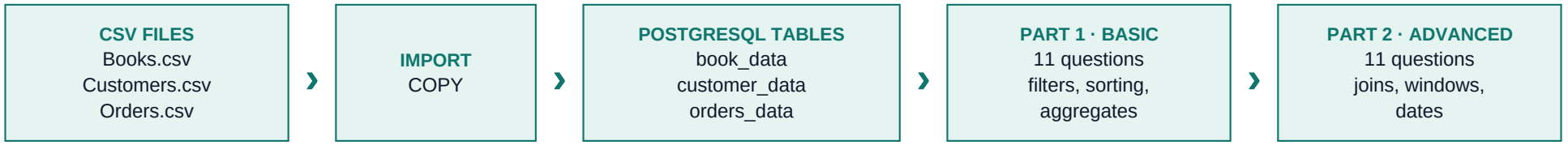
Project Overview

Analysing book sales with PostgreSQL across three related tables.

Objective

Answer business questions about books, customers and orders using SQL, first with basic queries and then with joins, grouping, window functions and date reporting.

Workflow



Two stages

	Part 1 · Basic Analysis	Part 2 · Advanced Analysis
Questions	11 questions, 12 queries	11 questions, 13 queries
Focus	Filtering, sorting, simple aggregates	Joins, GROUP BY, window functions, dates

How to read this document

Element	Meaning
Code block with teal edge	Original SQL from the script, unchanged. Keywords are blue, functions purple, text values orange, comments green.
Table below the code	Added explanation: what it does, concepts used, result and notes.
Result*	Calculated from the CSV files with pandas. The SQL was not run in PostgreSQL, so re-run it to confirm.
Note	Points to be aware of, such as ties hidden by LIMIT 1.

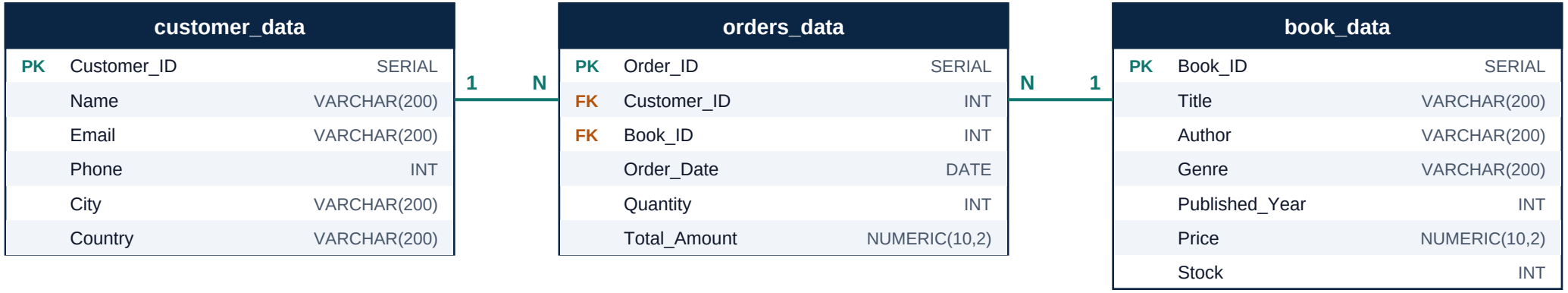
* Result values were calculated from the CSV files with pandas; the SQL was not run in PostgreSQL. Re-run to confirm.

Dataset and Database Structure

Three CSV files of 500 rows each, loaded into three related tables.

CSV file	Table	Rows	Contains
Books.csv	book_data	500	Title, author, genre, year, price, stock
Customers.csv	customer_data	500	Name, contact, city, country
Orders.csv	orders_data	500	Customer, book, date, quantity, total amount

Entity-relationship diagram



One customer can have many orders, and one book can appear in many orders.

* Result values were calculated from the CSV files with pandas; the SQL was not run in PostgreSQL. Re-run to confirm.

Table definitions

CREATE TABLE · book_data — **ORIGINAL · UNCHANGED**

```
CREATE TABLE book_data(
Book_ID SERIAL PRIMARY KEY,
Title VARCHAR(200),
Author VARCHAR(200),
Genre VARCHAR(200),
Published_Year INT,
Price NUMERIC(10,2),
Stock INT
);
```

CREATE TABLE · customer_data — **ORIGINAL · UNCHANGED**

```
CREATE TABLE customer_data(
Customer_ID SERIAL PRIMARY KEY,
Name VARCHAR(200),
Email VARCHAR(200),
Phone INT,
City VARCHAR(200),
Country VARCHAR(200)
);
```

CREATE TABLE · orders_data — **ORIGINAL · UNCHANGED**

```
CREATE TABLE orders_data(
Order_ID SERIAL PRIMARY KEY,
Customer_ID INT REFERENCES customer_data(Customer_ID),
Book_ID INT REFERENCES book_data(Book_ID),
Order_Date DATE,
Quantity INT,
Total_Amount NUMERIC(10,2)
);
```

PART 1

Basic SQL Analysis

11 questions · filtering, sorting and simple aggregates

Q00 Data import

COPY IMPORTS AND PREVIEWS

LINES 29-48 · ORIGINAL · UNCHANGED

```
--INSERT BOOK DATA
COPY book_data(Book_ID,Title,Author,Genre,Published_Year,Price,Stock)
FROM 'C:\Users\User\Downloads\SQL-13.9\Book Sales Analysis\Books.csv'
CSV HEADER;
SELECT * FROM book_data;

--INSERT CUSTMER DATA
COPY customer_data(Customer_ID,Name,Email,Phone,City,Country)
FROM 'C:\Users\User\Downloads\SQL-13.9\Book Sales Analysis\Customers.csv'
CSV HEADER;
SELECT * FROM customer_data;

--INSERT ORDER DATA
COPY orders_data(Order_ID,Customer_ID,Book_ID,Order_Date,Quantity,Total_Amount)
FROM 'C:\Users\User\Downloads\SQL-13.9\Book Sales Analysis\Orders.csv'
CSV HEADER;
SELECT * FROM orders_data;
```

EXPLAINS	Loads each CSV with COPY ... CSV HEADER.
NOTE	Absolute Windows paths on the database server: they work only on the author's machine.

Q1 Books in the Fiction genre

SQL

ORIGINAL · UNCHANGED

```sql
--Q1. Retrive all books in the 'Fiction' Genre:
SELECT * FROM book_data
WHERE Genre='Fiction';
```

EXPLAINS	All books where Genre is exactly 'Fiction'.
CONCEPTS	SELECT *, WHERE
RESULT*	60 books
NOTE	'Science Fiction' and 'Non-Fiction' are different values and are excluded.

Q2 Books published after 1950

SQL

ORIGINAL · UNCHANGED

```sql
--Q2. Find books published after the year 1950:
SELECT * FROM book_data
WHERE Published_Year > 1950 ORDER BY Published_Year ASC;
```

EXPLAINS	Books after 1950, oldest first.
CONCEPTS	WHERE, ORDER BY
RESULT*	292 books (1951-2023)

Q3 Customers in Canada

SQL

ORIGINAL · UNCHANGED

```sql
--Q3. List of all customer country 'CANADA':
SELECT * FROM customer_data
WHERE Country='Canada';
```

EXPLAINS	Customers whose Country is 'Canada'.
CONCEPTS	WHERE, case-sensitive text match
RESULT*	3 customers (IDs 38, 415, 468)
NOTE	Heading says 'CANADA'; the condition uses 'Canada', which matches the stored value.

Q4 Orders in November 2023

SQL

ORIGINAL · UNCHANGED

```sql
--Q4. Show orders placed in November 2023:
SELECT * FROM orders_data
WHERE
Order_Date BETWEEN '2023-11-01' AND '2023-11-30'
ORDER BY Order_Date ASC;
```

EXPLAINS	Orders dated 1-30 November 2023, in date order.
CONCEPTS	BETWEEN, ORDER BY
RESULT*	25 orders

* Result values were calculated from the CSV files with pandas; the SQL was not run in PostgreSQL. Re-run to confirm.

Q5 Total stock available

SQL — ORIGINAL · UNCHANGED

```sql
--Q5. Retrive total 'Stock' of book available:
SELECT SUM(stock) AS Total_Stock
FROM book_data;
```

EXPLAINS	Sum of Stock across all books.
CONCEPTS	SUM, alias
RESULT*	25,056 units

Q6 Most expensive book

SQL — ORIGINAL · UNCHANGED

```sql
--Q6. Find the details of the most expensive book:
SELECT * FROM book_data
ORDER BY Price DESC LIMIT 1;
```

EXPLAINS	Sorts by price descending and keeps one row.
CONCEPTS	ORDER BY DESC, LIMIT
RESULT*	Book_ID 340, 'Proactive system-worthy orchestration', 49.98

* Result values were calculated from the CSV files with pandas; the SQL was not run in PostgreSQL. Re-run to confirm.

Q7 Customers who ordered more than 1 copy

SQL — ORIGINAL · UNCHANGED

```sql
--Q7. Show all customers details who order more than 1 qunatiry of a book:
SELECT
    c.Customer_ID,c.Name, c.Email, c.Phone, c.Country,
    o.Book_ID,o.Order_Date,o.Quantity
FROM customer_data c
LEFT JOIN orders_data o
ON c.Customer_ID=o.Customer_ID
WHERE Quantity>1
ORDER BY Quantity DESC;
```

EXPLAINS	Joins customers to orders and keeps orders with Quantity > 1.
CONCEPTS	LEFT JOIN, WHERE, ORDER BY
RESULT*	438 order rows from 281 customers
NOTE	The WHERE filter makes the LEFT JOIN act as an INNER JOIN. A customer appears once per qualifying order.

Q8 Orders above $20

SQL — ORIGINAL · UNCHANGED

```sql
--Q8.Retrive all orders where the total amount exceeds $20;
SELECT * FROM orders_data
WHERE total_amount>20
ORDER BY total_amount ASC;
```

EXPLAINS	Orders with Total_Amount above 20, smallest first.
CONCEPTS	WHERE, ORDER BY
RESULT*	473 of 500 orders

* Result values were calculated from the CSV files with pandas; the SQL was not run in PostgreSQL. Re-run to confirm.

Q9 Genres in the Books table

METHOD 1 · GROUP BY — **ORIGINAL · UNCHANGED**

```sql
--Q9. List all genres available in the Books Table:
SELECT Genre AS ListOf_Genre
FROM book_data
GROUP BY Genre ORDER BY Genre ASC;
```

ALTERNATIVE · DISTINCT — **ORIGINAL · UNCHANGED**

```sql
--OR
SELECT DISTINCT(Genre) AS ListOf_Genre
FROM book_data ORDER BY Genre ASC;
```

EXPLAINS	Both return each genre once, alphabetically.
CONCEPTS	GROUP BY, DISTINCT
RESULT*	7 genres: Biography, Fantasy, Fiction, Mystery, Non-Fiction, Romance, Science Fiction

Q10 Book with the lowest stock

SQL — **ORIGINAL · UNCHANGED**

```sql
--Q10. Find the book with the lowest stock:
SELECT * FROM book_data
ORDER BY stock ASC LIMIT 1;
```

EXPLAINS	Sorts by stock ascending and keeps one row.
CONCEPTS	ORDER BY ASC, LIMIT
RESULT*	Lowest stock is 0, shared by 5 books
NOTE	LIMIT 1 returns only one of the tied books.

* Result values were calculated from the CSV files with pandas; the SQL was not run in PostgreSQL. Re-run to confirm.

Q11 Total revenue

SQL **ORIGINAL · UNCHANGED**

```sql
--Q11. Calculate the total revenue generated from all orders:
SELECT
SUM(total_amount) AS Total_Revenue
FROM orders_data;
```

EXPLAINS	Sum of Total_Amount over all orders.
CONCEPTS	SUM, alias
RESULT*	75,628.66 (revenue, not profit)

* Result values were calculated from the CSV files with pandas; the SQL was not run in PostgreSQL. Re-run to confirm.

PART 2

Advanced SQL Analysis

11 questions · joins, grouping, window functions and dates

Q1 Total books sold per genre

SQL · --ADVANCE SECTION-- (LINE 108) ORIGINAL · UNCHANGED

```sql
--ADVANCE SECTION--
--Q1. Retrive the total number of books sold for each genre:
SELECT
    b.Genre, SUM(o.Quantity) AS TotalQty_Sold
FROM book_data b
INNER JOIN orders_data o
ON b.Book_ID=o.Book_ID
GROUP BY Genre ORDER BY SUM(o.Quantity) DESC;
```

LOGIC	Joins books to orders, sums Quantity per genre, highest first.
CONCEPTS	INNER JOIN, GROUP BY, SUM
PURPOSE	Which genres sell the most copies.
RESULT*	Mystery 504 · Science Fiction 447 · Fantasy 446 · Romance 439 · Non-Fiction 351 · Biography 285 · Fiction 225

* Result values were calculated from the CSV files with pandas; the SQL was not run in PostgreSQL. Re-run to confirm.

Q2 Average price of Fantasy books

METHOD 1 · WINDOW FUNCTION — ORIGINAL · UNCHANGED

```sql
--Q2. Find the Average price of books in the 'Fantasy' Genre:
SELECT Title, Genre, price,
ROUND(AVG(Price) OVER(PARTITION BY Genre),2) AS Avg_Price
FROM book_data WHERE Genre='Fantasy';
```

ALTERNATIVE · PLAIN AGGREGATE — ORIGINAL · UNCHANGED

```sql
--Another Way
SELECT ROUND(AVG(Price),2) AS Avg_Price
FROM book_data WHERE Genre='Fantasy';
```

Compare	Method 1 · Window	Method 2 · Aggregate
Returns	One row per Fantasy book, average repeated	A single row
Use when	Showing each book beside the average	Only the average is needed

LOGIC	Both give the Fantasy average price, rounded to 2 decimals.
CONCEPTS	AVG() OVER, PARTITION BY, ROUND
PURPOSE	Benchmark Fantasy pricing.
RESULT*	25.98 across 71 books

* Result values were calculated from the CSV files with pandas; the SQL was not run in PostgreSQL. Re-run to confirm.

Q3 Customers with at least 2 orders

METHOD 1 · GROUP BY + HAVING — **ORIGINAL · UNCHANGED**

```sql
--Q3. List of customers who have placed at least 2 orders:
SELECT c.Customer_id, c.Name,
COUNT(o.Order_ID) AS NumberOf_Orders
FROM customer_data c
INNER JOIN orders_data o
ON c.Customer_id=o.Customer_id
GROUP BY c.Customer_id
HAVING COUNT(o.Order_ID)>=2
ORDER BY COUNT(o.Customer_id) DESC;
```

ALTERNATIVE · WINDOW FUNCTIONS — **ORIGINAL · UNCHANGED**

```sql
--Another way BY Window Function
SELECT Customer_id, Name, NumberOf_Orders FROM(
SELECT c.Customer_id, c.Name,
COUNT(o.Order_ID)
OVER(PARTITION BY o.customer_id) AS NumberOf_Orders,
ROW_NUMBER()
OVER(PARTITION BY o.customer_id ORDER BY o.customer_id ASC) AS EachCustomer_Row
FROM customer_data c
INNER JOIN orders_data o
ON c.Customer_id=o.Customer_id
) AS RPT_ANALYSIS
WHERE NumberOf_Orders>=2 AND EachCustomer_Row=1 ORDER BY NumberOf_Orders DESC;
```

Q3 · Customers with at least 2 orders (explanation)

Compare	Method 1 · GROUP BY	Method 2 · Window
Approach	One row per customer, filtered with HAVING	COUNT() OVER, then ROW_NUMBER() = 1 per customer
Style	Shorter, direct	Longer, shows PARTITION BY

LOGIC	Both list customers with two or more orders, most orders first.
CONCEPTS	COUNT, HAVING, PARTITION BY, ROW_NUMBER, subquery
PURPOSE	Identify repeat customers.
RESULT*	139 customers; most orders by one customer: 6 (ID 364)

Q4 Most frequently ordered book

SQL — ORIGINAL · UNCHANGED

```sql
--Q4. Find the most frequently orderd book:
SELECT b.Book_ID, b.Title,
COUNT(o.Order_ID) AS Order_Count
FROM book_data b
INNER JOIN orders_data o
ON b.Book_ID=o.Book_ID
GROUP BY b.Book_ID
ORDER BY COUNT(o.Order_ID) DESC LIMIT 1;
```

LOGIC	Counts order rows per book and returns the highest.
CONCEPTS	INNER JOIN, COUNT, GROUP BY, LIMIT
PURPOSE	Most popular book by number of orders.
RESULT*	Highest count is 4 orders
NOTE	7 books tie at 4; LIMIT 1 returns only one.

* Result values were calculated from the CSV files with pandas; the SQL was not run in PostgreSQL. Re-run to confirm.

Q5 Top 3 Fantasy books by price

SQL **ORIGINAL · UNCHANGED**

```sql
--Q5. Show the top 3 most expensive books of 'Fantasy' Genre:
SELECT
Book_ID, Title, Genre, Price
FROM book_data
WHERE Genre='Fantasy'
ORDER BY Price DESC LIMIT 3;
```

LOGIC	Fantasy books sorted by price, top three kept.
CONCEPTS	WHERE, ORDER BY DESC, LIMIT
PURPOSE	Premium end of the Fantasy range.
RESULT*	Book_ID 240 (49.90) · 462 (49.23) · 238 (48.97)

* Result values were calculated from the CSV files with pandas; the SQL was not run in PostgreSQL. Re-run to confirm.

Q6 Total quantity sold by author

SQL **ORIGINAL · UNCHANGED**

```sql
--Q6. Retrive the total quantity of books sold by each author:
SELECT b.Author, SUM(o.Quantity) AS Total_Books_Sold
FROM book_data b
INNER JOIN orders_data o
ON b.Book_ID=o.Book_ID
GROUP BY b.Author ORDER BY b.Author ASC;
```

LOGIC	Sums Quantity per author, alphabetical order.
CONCEPTS	INNER JOIN, GROUP BY, SUM
PURPOSE	Copies sold per author.
RESULT*	314 authors; top: Patrick Contreras (28 copies)
NOTE	Authors without orders are excluded by the INNER JOIN.

* Result values were calculated from the CSV files with pandas; the SQL was not run in PostgreSQL. Re-run to confirm.

Q7 Customers who spent over $30

SQL

ORIGINAL · UNCHANGED

```sql
--Q7. List the cities where customers who spent over $30 are located:
SELECT c.Customer_ID, c.City,
SUM(o.Total_Amount) AS Total_Spend
FROM customer_data c
INNER JOIN orders_data o
ON c.Customer_id=o.Customer_id
GROUP BY c.Customer_ID, c.City
HAVING SUM(o.Total_Amount) >30 ORDER BY Customer_ID ASC;
```

LOGIC	Totals spending per customer and keeps totals above 30, with city.
CONCEPTS	GROUP BY, SUM, HAVING
PURPOSE	Higher-spending customers and their cities.
RESULT*	286 of 307 customers with orders
NOTE	One row per customer, so a city can repeat.

* Result values were calculated from the CSV files with pandas; the SQL was not run in PostgreSQL. Re-run to confirm.

Q8 Customer who spent the most

SQL **ORIGINAL · UNCHANGED**

```sql
--Q8. Find the customer who spent the most on orders:
SELECT c.Customer_ID, c.Name,
SUM(o.Total_Amount) AS Total_Spent
FROM customer_data c
INNER JOIN orders_data o
ON c.Customer_id=o.Customer_id
GROUP BY c.Customer_ID
ORDER BY SUM(o.Total_Amount) DESC LIMIT 1;
```

LOGIC	Totals spending per customer and returns the top one.
CONCEPTS	GROUP BY, SUM, ORDER BY, LIMIT
PURPOSE	Identify the top customer.
RESULT*	Customer_ID 457: 1,398.90 (next: ID 174, 1,080.95)

* Result values were calculated from the CSV files with pandas; the SQL was not run in PostgreSQL. Re-run to confirm.

Q9 Stock remaining after all orders

SQL **ORIGINAL · UNCHANGED**

```sql
--Q9.Calculate the stock remaining after fulfilling all order:
SELECT b.Book_ID, SUM(b.stock) AS Total_Stock,
SUM(o.Quantity) AS Sales_Qty,
SUM(b.stock)-COALESCE(SUM(o.Quantity),0) AS Remaining_Stock
FROM book_data b
LEFT JOIN orders_data o
ON b.Book_ID=o.Book_ID
GROUP BY b.Book_ID ORDER BY b.Book_ID ASC;
```

LOGIC	Left-joins books to orders and subtracts quantity sold from stock; COALESCE treats no sales as 0.
CONCEPTS	LEFT JOIN, SUM, COALESCE
PURPOSE	Estimate remaining inventory.
RESULT*	500 books; 183 have no orders
NOTE	SUM(b.stock) runs after the join, so stock is added once per order row and is overstated for books with 2+ orders. Example: Book_ID 3 (stock 27, 5 sold) shows 49 remaining instead of 22. Affects 135 books. Original code left unchanged.

* Result values were calculated from the CSV files with pandas; the SQL was not run in PostgreSQL. Re-run to confirm.

Q10 Monthly revenue and copies sold

SQL

ORIGINAL · UNCHANGED

```sql
--Q10. Analyze monthly sales revenue and the total number of books sold for each month.
SELECT
EXTRACT(YEAR FROM order_date) AS Sales_Year,
EXTRACT(MONTH FROM order_date) AS Sales_Month,
SUM(quantity) AS Sales_Qty,
SUM(total_amount) AS Total_Revenue
FROM orders_data
GROUP BY
EXTRACT(YEAR FROM order_date),
EXTRACT(MONTH FROM order_date)
ORDER BY
EXTRACT(YEAR FROM order_date),
EXTRACT(MONTH FROM order_date) ASC;
```

LOGIC	Groups orders by year and month; sums quantity and revenue.
CONCEPTS	EXTRACT, GROUP BY, SUM
PURPOSE	Monthly sales trend.
RESULT*	25 months: Dec 2022 (79 copies, 2,513.36) to Dec 2024 (29 copies, 865.58)
NOTE	First and last months are partial (data runs 9 Dec 2022 - 7 Dec 2024).

* Result values were calculated from the CSV files with pandas; the SQL was not run in PostgreSQL. Re-run to confirm.

Q11 Rank customers by spending

SQL · END OF SCRIPT, LINES 230-231 ORIGINAL · UNCHANGED

```sql
--Q11. Rank customers based on their total spending on orders.
SELECT
    Customer_ID, Name, Total_Spend,
DENSE_RANK() OVER(ORDER BY Total_Spend DESC) AS Customer_Rank
FROM(
SELECT
    c.customer_id, c.name,
SUM(o.total_amount) AS Total_Spend
FROM customer_data c
INNER JOIN orders_data o
ON c.customer_id=o.customer_id
GROUP BY c.customer_id
    ) AS Summary
ORDER BY Customer_Rank ASC;
---END---
```

LOGIC	Inner query totals spend per customer; DENSE_RANK ranks them and ties share a rank.
CONCEPTS	Subquery, DENSE_RANK() OVER
PURPOSE	Customer spending leaderboard.
RESULT*	307 customers, ranks 1-300; rank 1 is ID 457

* Result values were calculated from the CSV files with pandas; the SQL was not run in PostgreSQL. Re-run to confirm.

Conclusion and Technical Notes

What the project covers, and points to be aware of.

SQL skills demonstrated

Area	Used
Schema	CREATE TABLE, SERIAL, primary and foreign keys, data types
Loading	COPY ... CSV HEADER
Retrieval	WHERE, BETWEEN, ORDER BY, LIMIT, DISTINCT
Aggregation	SUM, COUNT, AVG, ROUND, GROUP BY, HAVING, COALESCE
Joins and subqueries	INNER JOIN, LEFT JOIN, subquery in FROM
Window functions	AVG OVER, COUNT OVER, ROW_NUMBER, DENSE_RANK
Dates	EXTRACT(YEAR / MONTH)

Coverage

Item	Count
CREATE TABLE statements	3
COPY imports + preview SELECT *	3 + 3
Basic: questions / queries	11 / 12
Advanced: questions / queries	11 / 13

Technical notes

- **Results.** Result* values were computed with pandas, not in PostgreSQL. Amounts are revenue, not profit.
- **Advanced Q9.** Stock is multiplied by the number of order rows per book (see the note on that query).
- **Ties.** LIMIT 1 hides ties in Basic Q10 and Advanced Q4.
- **Import.** COPY uses server-side Windows paths. CSV dates are M/D/YYYY, which loads under the default date style.
- **Data types.** Phone is INT, which fits this data but not longer numbers. SERIAL sequences are not advanced by COPY with explicit IDs.
- **Presentation.** Original comments, including their spelling, are kept as written. Tabs are shown as four spaces.

* Result values were calculated from the CSV files with pandas; the SQL was not run in PostgreSQL. Re-run to confirm.